Calidad y Preprocesamiento de Datos

MCIC VÍCTOR MANUEL CORZA VARGAS

MC FERNANDO AVITÚA VARELA

Unidad 5 Transformación y Preprocesamiento de Datos

Objetivo de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Identificar la necesidad de transformar los datos durante su integración
- Aplicar técnicas fundamentales como la normalización y el escalado
- Preparar los datos para análisis posteriores
- Comprender fundamentos matemáticos y casos de uso de las diferentes técnicas de preprocesamiento y transformación de datos

5.1 Transformación durante la integración de datos

La integración de datos consiste en combinar información proveniente de múltiples fuentes (bases de datos, archivos, APIs, etc.) en una vista unificada.

Estas fuentes a menudo presentan heterogeneidad: diferentes esquemas, formatos, unidades de medida, convenciones de codificación y, por supuesto, diferentes niveles de calidad.

La transformación de datos durante este proceso es fundamental para resolver estas inconsistencias y asegurar que el conjunto de datos resultante sea coherente, útil y preciso.

¿Qué es la transformación en este contexto?

Se refiere al conjunto de operaciones aplicadas a los datos extraídos de las fuentes originales antes o mientras se cargan en el destino final (por ejemplo, un Data Warehouse, un DataMart o simplemente un conjunto de datos consolidado para análisis). El objetivo es convertir los datos a un formato y estructura estandarizados y consistentes.

¿Por qué es necesaria la transformación durante la integración?

Resolver Heterogeneidad Estructural:

Diferencias de Esquema: Mapear nombres de columnas (ej. Cliente_ID vs CustomerID), ajustar tipos de datos (ej. string a integer), reordenar columnas.

Diferencias en Modelos de Datos: Convertir datos de formatos no relacionales a relacionales, o viceversa; aplanar estructuras jerárquicas.

¿Por qué es necesaria la transformación durante la integración?

Resolver Heterogeneidad Semántica:

Unidades de Medida: Convertir valores (ej. de libras a kilogramos, de Fahrenheit a Celsius).

Codificaciones y Representaciones: Estandarizar valores categóricos (ej. M/F, Hombre/Mujer, 1/0 todos a Masculino/Femenino), formatos de fecha (DD/MM/YYYY vs MM-DD-YY),etc.

Niveles de Agregación: Asegurar que los datos estén al mismo nivel de detalle (ej. ventas diarias vs ventas mensuales).

¿Por qué es necesaria la transformación durante la integración?

Limpieza y Estandarización de Valores:

Aplicar reglas de limpieza definidas previamente (ej. corregir errores tipográficos comunes).

Resolver inconsistencias que sólo son aparentes al comparar fuentes (ej. un cliente con dos direcciones diferentes en dos sistemas).

¿Por qué es necesaria la transformación durante la integración?

Adecuación para el Análisis:

Crear nuevas variables calculadas (ej. edad a partir de fecha_nacimiento).

Aplicar técnicas como la **normalización y escalado** para que los atributos numéricos sean comparables o cumplan los requisitos de ciertos algoritmos.

Retos comunes

- Identificar todas las inconsistencias necesarias a transformar.
- Definir las reglas de transformación correctas y robustas.
- Manejar grandes volúmenes de datos eficientemente durante la transformación.
- Asegurar que la transformación no introduzca nuevos errores o sesgos.

5.1.1 Normalización y escalado

Dentro de las transformaciones numéricas, la normalización y el escalado son técnicas esenciales, especialmente en el contexto del Machine Learning y el análisis estadístico.

5.1.1 Normalización y escalado

Muchos algoritmos son sensibles a la escala de las características de entrada. Por ejemplo, algoritmos basados en distancias (como **k-NN, K-Means, SVM**) o aquellos que utilizan descenso de gradiente (como **regresión lineal, redes neuronales**) pueden dar un peso desproporcionado a las características con rangos de valores más grandes, o pueden tener problemas de convergencia si las escalas son muy diferentes.

La normalización y el escalado buscan mitigar este problema ajustando el rango y/o la distribución de las variables numéricas.

Escalado

Escalado (Scaling): Término general que se refiere a cambiar el rango de los datos. No implica necesariamente un rango específico ni una distribución particular.

Métodos principales de escalado

Normalización (Normalization): A menudo se refiere específicamente a escalar los datos a un rango fijo, comúnmente [0, 1] o [-1, 1]. El método más común para esto es el **Escalado Min-Max**.

Estandarización (Standardization): Técnica específica de escalado que transforma los datos para que tengan una media (μ) de 0 y una desviación estándar (σ) de 1. También se conoce como **Escalado Z-score**.

Normalización Min-Max (Min-Max Scaling):

Objetivo: Escalar los datos a un rango fijo, típicamente [0, 1].

Fórmula: Para una característica X, cada valor x se transforma en x′ usando:

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

donde min(X) y max(X) son los valores mínimo y máximo de la característica X en el conjunto de datos (o en el conjunto de entrenamiento).

Rango Resultante: [0, 1].

Normalización Min-Max (Min-Max Scaling):

Ventajas: Garantiza que todos los valores estén dentro de un rango acotado. Útil para algoritmos que requieren entradas en este rango (ej. algunas redes neuronales, procesamiento de imágenes donde los pixeles van de 0 a 255, que se puede escalar a 0-1).

Desventajas: Muy sensible a valores atípicos (outliers). Un solo valor extremo puede comprimir la mayoría de los datos en un rango muy pequeño dentro de [0, 1], perdiendo información relativa entre ellos.

Estandarización Z-score Scaling:

Objetivo: Transformar los datos para que tengan media 0 y desviación estándar 1.

Fórmula: Para una característica X, cada valor x se transforma en x′ usando:

$$x' = \frac{x - \mu}{\sigma}$$

donde μ es la media y σ es la desviación estándar de la característica X en el conjunto de datos (o en el conjunto de entrenamiento).

Rango Resultante: No está acotado a un rango fijo (los valores pueden ser positivos o negativos, y teóricamente ir de −∞ a +∞, aunque típicamente se concentran entre -3 y 3 si los datos originales se asemejan a una distribución normal).

Estandarización (Z-score Scaling / Standardization):

Ventajas: Menos sensible a outliers que Min-Max. Es el método preferido para muchos algoritmos de Machine Learning (PCA, regresión lineal/logística, SVM) que asumen que los datos están centrados en cero y/o tienen varianza unitaria. Ayuda a que los algoritmos de optimización (como el descenso de gradiente) converjan más rápido.

Desventajas: No confina los datos a un rango específico, lo cual podría ser un requisito para ciertos algoritmos.

¿Cuándo usar cuál?

Usa Normalización Min-Max si:

- Necesitas que los valores estén estrictamente en el rango [0, 1] (o [-1, 1]).
- Sabes que no hay (o has tratado) outliers significativos en tus datos.
- El algoritmo que vas a usar no hace suposiciones sobre la distribución de los datos (ej. k-NN, aunque la estandarización también funciona bien aquí).

¿Cuándo usar cuál?

Usa Estandarización (Z-score) si:

- El algoritmo asume que los datos están centrados en cero o tienen una distribución Gaussiana (aunque no es un requisito estricto, a menudo mejora el rendimiento).
- Hay outliers en los datos (es más robusta que Min-Max).
- Es el método "por defecto" o el más comúnmente recomendado para muchos algoritmos de ML (Regresión, SVM, PCA, Redes Neuronales).

Alternativa: **Escalado Robusto (Robust Scaling)**

Si los outliers son un problema significativo, se puede usar un escalador que utilice la mediana y el Rango Intercuartílico (IQR).

Fórmula:

$$x' = \frac{x - \text{mediana}(X)}{\text{IQR}(X)}$$

Es mucho menos sensible a valores extremos.

Ejemplo Práctico:

Supongamos que tenemos datos de edad y salario de 3 personas:

Persona 1: Edad = 25, Salario = 30000

Persona 2: Edad = 45, Salario = 70000

Persona 3: Edad = 60, Salario = 50000

Ejemplo Práctico:

Aplicando Normalización **Min-Max** (Rango [0, 1]):

Edad: Min=25, Max=60

Persona 1: $(25-25)/(60-25)=0/35=0$

Persona 2: $(45-25)/(60-25)=20/35\approx0.571$

Persona 3: $(60-25)/(60-25)=35/35=1$

Salario: Min=30000, Max=70000

Persona 1: $(30000-30000)/(70000-30000)=0/40000=0$

Persona 2: $(70000-30000)/(70000-30000)=40000/40000=1$

Persona 3: $(50000-30000)/(70000-30000)=20000/40000=0.5$

Ejemplo Práctico:

Aplicando Estandarización (Z-score):

Edad: Media(μ) = (25+45+60)/3 = 130/3 ≈ 43.33. Desv. Estándar(σ) ≈ 17.56

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

Persona 1: (25−43.33)/17.56≈−1.044

Persona 2: (45−43.33)/17.56≈0.095

Persona 3: (60−43.33)/17.56≈0.949

Salario: Media(μ) = (30000+70000+50000)/3 = 150000/3 = 50000. Desv. Estándar(σ) ≈ 16330 (usando desviación estándar poblacional)

Persona 1: (30000−50000)/16330≈−1.225

Persona 2: (70000−50000)/16330≈1.225

Persona 3: (50000−50000)/16330=0

Consideraciones Importantes en la Práctica

Fuga de Datos (Data Leakage):

La fuga de datos se produce cuando la información del conjunto de pruebas influye involuntariamente en el proceso de entrenamiento, lo que conduce a un rendimiento excesivamente optimista del modelo. Para mitigar este riesgo en el proceso de transformación de datos es necesario tomar en cuenta las siguientes consideraciones:

Al preparar datos para Machine Learning, es crucial calcular los parámetros de escalado (min, max, media, desviación estándar) únicamente sobre el conjunto de entrenamiento (training set).

Luego, estos mismos parámetros se usan para transformar tanto el conjunto de entrenamiento como el conjunto de prueba (test set) y cualquier dato nuevo futuro.

Nunca calcules los parámetros usando el conjunto de prueba o todo el dataset completo antes de dividirlo.